

Frames are an artificial intelligence data structure used to divide knowledge into substructures by representing "stereotyped situations".

They were proposed by Marvin Minsky in his 1974 article "A Framework for Representing Knowledge". Frames are the primary data structure used in artificial intelligence frame languages; they are stored as ontologies of sets.

Frames are also an extensive part of knowledge representation and reasoning schemes. They were originally derived from semantic networks and are therefore part of structure-based knowledge representations.

According to Russell and Norvig's Artificial Intelligence: A Modern Approach, structural representations assemble "facts about particular object and event types and [arrange] the types into a large taxonomic hierarchy analogous to a biological taxonomy".

Frame structure

The frame contains information on how to use the frame, what to expect next, and what to do when these expectations are not met.

Some information in the frame is generally unchanged while other information, stored in "terminals", usually change. Terminals can be considered as variables.

Top-level frames carry information, that is always true about the problem in hand, however, terminals do not have to be true. Their value might change with the new information encountered. Different frames may share the same terminals.

Each piece of information about a particular frame is held in a slot.

The information can contain:

Facts or Data

Values (called facets)

Procedures (also called procedural attachments)

IF-NEEDED: deferred evaluation

IF-ADDED: updates linked information

Default Values

For Data

For Procedures

Other Frames or Subframes

Features and advantages

A frame's terminals are already filled with default values, which is based on how the human mind works.

For example, when a person is told "a boy kicks a ball", most people will visualize a particular ball (such as a familiar soccer ball) rather than imagining some abstract ball with no attributes.

One particular strength of frame-based knowledge representations is that, unlike semantic networks, they allow for exceptions in particular instances. This gives frames a degree of flexibility that allows representations to reflect real-world phenomena more accurately.

Like semantic networks, frames can be queried using spreading activation. Following the rules of inheritance, any value given to a slot that is inherited by subframes will be updated (IF-ADDED) to the corresponding slots in the subframes and any new instances of a particular frame will feature that new value as the default.

Because frames are based on structures, it is possible to generate a semantic network given a set of frames even though it lacks explicit arcs. References to Noam Chomsky and his generative grammar of 1950 are generally missing from Minsky's work.

The simplified structures of frames allow for easy analogical reasoning, a much prized feature in any intelligent agent. The procedural attachments provided by frames also allow a degree of flexibility that makes for a more realistic representation and gives a natural affordance for programming applications.

Example

Worth noticing here is the easy analogical reasoning (comparison) that can be done between a boy and a monkey just by having similarly named slots.

Also notice that Alex, an instance of a boy, inherits default values like "Sex" from the more general parent object Boy,

but the boy may also have different instance values in the form of exceptions such as the number of legs.

Frame language

A frame language is a technology used for knowledge representation in artificial intelligence. They are similar to class hierarchies in object-oriented languages although their fundamental design goals are different.

Frames are focused on explicit and intuitive representation of knowledge whereas objects focus on encapsulation and information hiding. Frames originated in AI research and objects primarily in software engineering.

However, in practice, the techniques and capabilities of frame and object-oriented languages overlap significantly.

Example

A simple example of concepts modeled in a frame language is the Friend of A Friend (FOAF) ontology defined as part of the Semantic Web as a foundation for social networking and calendar systems.

The primary frame in this simple example is a Person.

Example slots are the person's email, home page, phone, etc.

The interests of each person can be represented by additional frames describing the space of

business and entertainment domains.

The slot knows links each person with other persons.

Default values for a person's interests can be inferred by the web of people they are friends of.

Implementations

The earliest frame-based languages were custom developed for specific research projects and were not packaged as tools to be re-used by other researchers.

Just as with expert system inference engines, researchers soon realized the benefits of extracting part of the core infrastructure and developing general-purpose frame languages that were not coupled to specific applications.

One of the first general-purpose frame languages was KRL. One of the most influential early frame languages was KL-ONE. KL-ONE spawned several subsequent Frame languages.

One of the most widely used successors to KL-ONE was the Loom language developed by Robert MacGregor at the Information Sciences Institute.

In the 1980s, Artificial Intelligence generated a great deal of interest in the business world fueled by expert systems. This led to the development of many commercial products for the development of knowledge-based systems. These early products were usually developed in Lisp and integrated constructs such as IF-THEN rules for logical reasoning with Frame hierarchies for representing data.

One of the most well known of these early Lisp knowledge-base tools was the Knowledge Engineering Environment (KEE) from Intellicorp. KEE provided a full Frame language with multiple inheritance, slots, triggers, default values, and a rule engine that supported backward and forward chaining. As with most early commercial versions of AI software KEE was originally deployed in Lisp on Lisp machine platforms but was eventually ported to PCs and Unix workstations.

The research agenda of the Semantic Web spawned a renewed interest in automatic

classification and frame languages. An example is the Web Ontology Language (OWL) standard for describing information on the Internet. OWL is a standard to provide a semantic layer on top of the Internet. The goal is that rather than searching the web using keywords as most search engines (e.g. Google) do today, the web can be organized by concepts organized in an ontology, like a directory structure.

The name of the OWL language itself provides a good example of the value of a Semantic Web. If one were to search for "OWL" using the Internet today most of the pages retrieved would be on the bird Owl rather than the standard OWL. With a Semantic Web it would be possible to specify the concept "Web Ontology Language" and the user would not need to worry about the various possible acronyms or synonyms as part of the search. Likewise, the user would not need to worry about homonyms crowding the search results with irrelevant data such as information about birds of prey as in this simple example.

In addition to OWL, various standards and technologies that are relevant to the Semantic Web and were influenced by Frame languages include OIL and DAML. The Protege Open Source software tool from Stanford University provides an ontology editing capability that is built on OWL and has the full capabilities of a classifier. However it ceased to explicitly support frames as of version 3.5 (which is maintained for those preferring frame orientation), with the current version being 5.6.8 as of 2025. The justification for moving from explicit frames being that OWL DL is more expressive and "industry standard".

Comparison of frames and objects

Frame languages have a significant overlap with object-oriented languages. The terminologies and goals of the two communities were different but as they moved from the academic world and labs to the commercial world developers tended to not care about philosophical issues and focused primarily on specific capabilities, taking the best from either camp regardless of where the idea began. What both paradigms have in common is a desire to reduce the

distance between concepts in the real world and their implementation in software. As such both paradigms arrived at the idea of representing the primary software objects in taxonomies starting with very general types and progressing to more specific types.

The following table illustrates the correlation between standard terminology from the object-oriented and frame language communities:

The primary difference between the two paradigms was in the degree that encapsulation was considered a major requirement. For the object-oriented paradigm encapsulation was one of, if not the most, critical requirement. The desire to reduce the potential interactions between software components and hence manage large complex systems was a key driver of object-oriented technology. For the frame language camp this requirement was less critical than the desire to provide a vast array of possible tools to represent rules, constraints, and programming logic. In the object-oriented world everything is controlled by methods and the visibility of methods. So for example, accessing the data value of an object property must be done via an accessor method. This method controls things such as validating the data type and constraints on the value being retrieved or set on the property. In Frame languages these same types of constraints could be handled in multiple ways. Triggers could be defined to fire before or after a value was set or retrieved. Rules could be defined that managed the same types of constraints. The slots themselves could be augmented with additional information (called "facets" in some languages) again with the same type of constraint information.

The other main differentiator between frame and OO languages was multiple inheritance (allowing a frame or class to have two or more superclasses). For frame languages multiple inheritance was a requirement. This follows from the desire to model the world the way humans do, human conceptualizations of the world seldom fall into rigidly defined non-overlapping taxonomies. For many OO languages, especially in the later years of OO, single inheritance was either strongly desired or required. Multiple inheritance was seen as a

possible step in the analysis phase to model a domain but something that should be eliminated in the design and implementation phases in the name of maintaining encapsulation and modularity.

Although the early frame languages such as KRL did not include message passing, driven by the demands of developers, most of the later frame languages (e.g. Loom, KEE) included the ability to define messages on Frames.

On the object-oriented side, standards have also emerged that provide essentially the equivalent functionality that frame languages provided, albeit in a different format and all standardized on object libraries. For example, the Object Management Group has standardized specifications for capabilities such as associating test data and constraints with objects (analogous to common uses for facets in Frames and to constraints in Frame languages such as Loom) and for integrating rule engines.

History

Early work on Frames was inspired by psychological research going back to the 1930s that indicated people use stored stereotypical knowledge to interpret and act in new cognitive situations. The term Frame was first used by Marvin Minsky as a paradigm to understand visual reasoning and natural language processing. In these and many other types of problems the potential solution space for even the smallest problem is huge. For example, extracting the phonemes from a raw audio stream or detecting the edges of an object. Things that seem trivial to humans are actually quite complex. In fact, how difficult they really were was probably not fully understood until AI researchers began to investigate the complexity of getting computers to solve them.

The initial notion of Frames or Scripts as they were also called is that they would establish the context for a problem and in so doing automatically reduce the possible search space significantly. The idea was also adopted by Schank and Abelson who used it to illustrate how

an AI system could process common human interactions such as ordering a meal at a restaurant. These interactions were standardized as Frames with slots that stored relevant information about each Frame. Slots are analogous to object properties in object-oriented modeling and to relations in entity-relation models. Slots often had default values but also required further refinement as part of the execution of each instance of the scenario. I.e., the execution of a task such as ordering at a restaurant was controlled by starting with a basic instance of the Frame and then instantiating and refining various values as appropriate. Essentially the abstract Frame represented an object class and the frame instances an object instance. In this early work, the emphasis was primarily on the static data descriptions of the Frame. Various mechanisms were developed to define the range of a slot, default values, etc. However, even in these early systems there were procedural capabilities. One common technique was to use "triggers" (similar to the database concept of triggers) attached to slots. A trigger is simply procedural code that have attached to a slot. The trigger could fire either before and/or after a slot value was accessed or modified.

As with object classes, Frames were organized in subsumption hierarchies. For example, a basic frame might be ordering at a restaurant. An instance of that would be Joe goes to Dairy Queen. A specialization (essentially a subclass) of the restaurant frame would be a frame for ordering at a fancy restaurant. The fancy restaurant frame would inherit all the default values from the restaurant frame but also would either add more slots or change one or more of the default values (e.g., expected price range) for the specialized frame.

Languages

Much of the early Frame language research (e.g. Schank and Abelson) had been driven by findings from experimental psychology and attempts to design knowledge representation tools that corresponded to the patterns humans were thought to use to function in daily tasks. These researchers were less interested in mathematical formality since they believed such

formalisms were not necessarily good models for the way the average human conceptualizes the world. The way humans use language for example is often far from truly logical.

Similarly, in linguistics, Charles J. Fillmore in the mid-1970s started working on his theory of frame semantics, which later would lead to computational resources like FrameNet. Frame semantics was motivated by reflections on human language and human cognition.

Researchers such as Ron Brachman on the other hand wanted to give AI researchers the mathematical formalism and computational power that were associated with Logic. Their aim was to map the Frame classes, slots, constraints, and rules in a Frame language to set theory and logic. One of the benefits of this approach is that the validation and even creation of the models could be automated using theorem provers and other automated reasoning capabilities. The drawback was that it could be more difficult to initially specify the model in a language with a formal semantics.

This evolution also illustrates a classic divide in AI research known as the "neats vs. scruffies". The "neats" were researchers who placed the most value on mathematical precision and formalism which could be achieved via First Order Logic and Set Theory. The "scruffies" were more interested in modeling knowledge in representations that were intuitive and psychologically meaningful to humans.

The most notable of the more formal approaches was the KL-ONE language. KL-ONE later went on to spawn several subsequent Frame languages. The formal semantics of languages such as KL-ONE gave these frame languages a new type of automated reasoning capability known as the classifier. The classifier is an engine that analyzes the various declarations in the frame language: the definition of sets, subsets, relations, etc. The classifier can then automatically deduce various additional relations and can detect when some parts of a model are inconsistent with each other. In this way many of the tasks that would normally be executed by forward or backward chaining in an inference engine can instead be performed by the classifier.

This technology is especially valuable in dealing with the Internet. It is an interesting result that the formalism of languages such as KL-ONE can be most useful dealing with the highly informal and unstructured data found on the Internet. On the Internet it is simply not feasible to require all systems to standardize on one data model. It is inevitable that terminology will be used in multiple inconsistent forms. The automatic classification capability of the classifier engine provides AI developers with a powerful toolbox to help bring order and consistency to a very inconsistent collection of data (i.e., the Internet). The vision for an enhanced Internet, where pages are ordered not just by text keywords but by classification of concepts is known as the Semantic Web. Classification technology originally developed for Frame languages is a key enabler of the Semantic Web. The "neats vs. scruffies" divide also emerged in Semantic Web research, culminating in the creation of the Linking Open Data community—their focus was on exposing data on the Web rather than modeling.

See also

Frame problem

Deductive classifier

Description logic

First-order logic

Knowledge base

Knowledge-based system

Knowledge graph

Ontology language

Predicate

Semantic network

Situation calculus

References

Bibliography

Russell, Stuart J.; Norvig, Peter (2010), *Artificial Intelligence: A Modern Approach* (2nd ed.), Upper Saddle River, New Jersey: Prentice Hall, ISBN 0-13-604259-7, ch. 1.

Marvin Minsky, *A Framework for Representing Knowledge* Archived 2021-01-07 at the Wayback Machine, MIT-AI Laboratory Memo 306, June, 1974.

Daniel G. Bobrow, Terry Winograd, *An Overview of KRL, A Knowledge Representation Language*

R. Bruce Roberts and Ira P. Goldstein, *The FRL Primer*, 1977

R. Bruce Roberts and Ira P. Goldstein, *The FRL Manual*, 1977

Brachman, R.; Schmolze, J. (1985). "An overview of the KL-ONE Knowledge Representation System". *Cognitive Science*. 9 (2): 171–216. doi:10.1016/s0364-0213(85)80014-8.

Fikes, R. E.; Kehler, T. (1985). "The role of frame-based representation in knowledge representation and reasoning". *Communications of the ACM*. 28 (9): 904–920. doi:10.1145/4284.4285. S2CID 9868560.

Peter Clark & Bruce Porter: *KM - The Knowledge Machine 2.0: Users Manual*, <http://www.cs.utexas.edu/users/mfkb/RKF/km.html>.

Peter D. Karp, *The Design Space of Frame Knowledge Representation Systems*, Technical Note 520. Artificial Intelligence Center, SRI International, 1992

External links

Minsky's "A Framework for Representing Knowledge" Archived 2021-01-07 at the Wayback Machine

Artificial Intelligence: A Modern Approach Website

Frame-Based Systems Archived 2009-07-27 at the Wayback Machine

The Generic Frame Protocol

The Protégé Ontology Editor

Intro Presentation to Frame Languages